Modeling Data Records

CS412
Boston University

Aaron Stevens

BOSTON UNIVERSITY

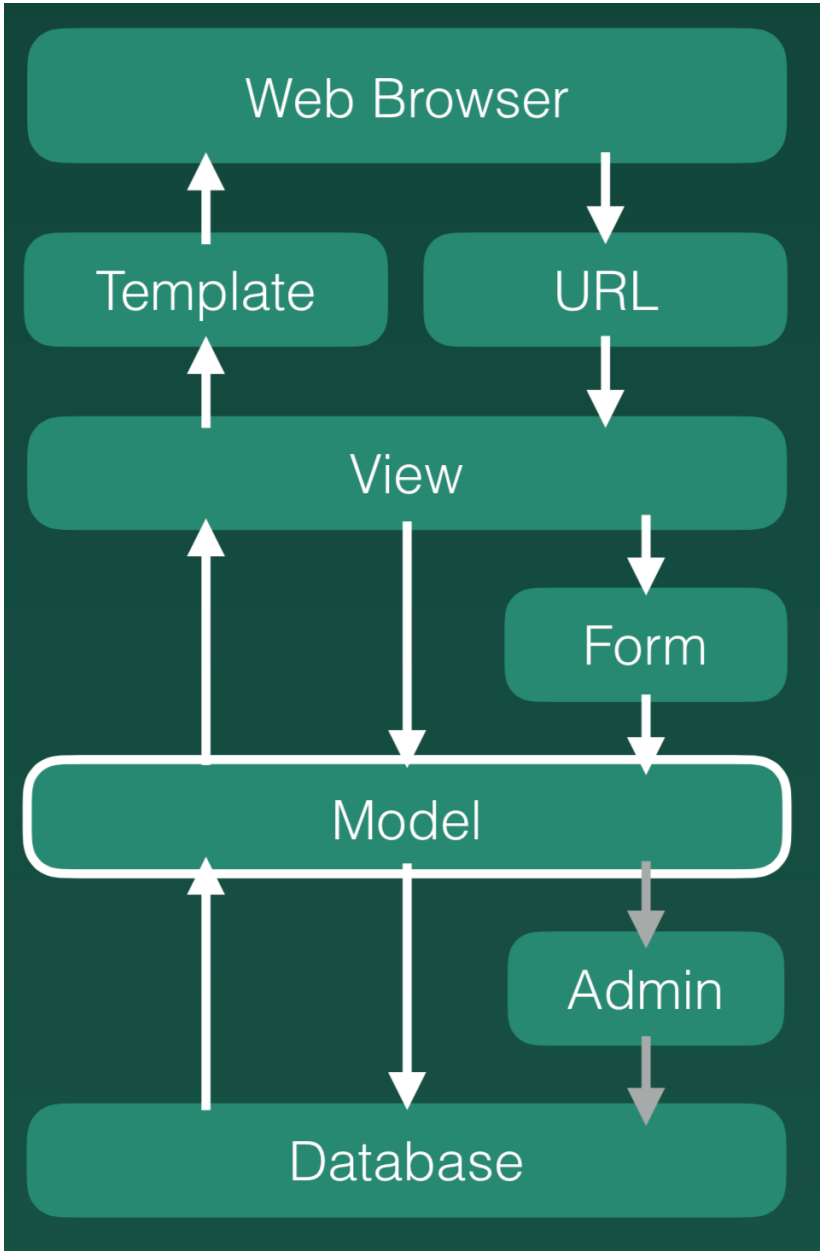
Learning Objectives

- Define a Django Model class with appropriate field types and options
- Run **`makemigrations`** and **`migrate`** to apply model changes to the database
- Use the Django Admin interface to add, view, and manage model data
- Implement **`__str__()`** on a model to give objects human-readable representations
- Use Django generic views (**`ListView`**, **`DetailView`**) to display model data in templates

Recall: The Model-View-Controller Architecture

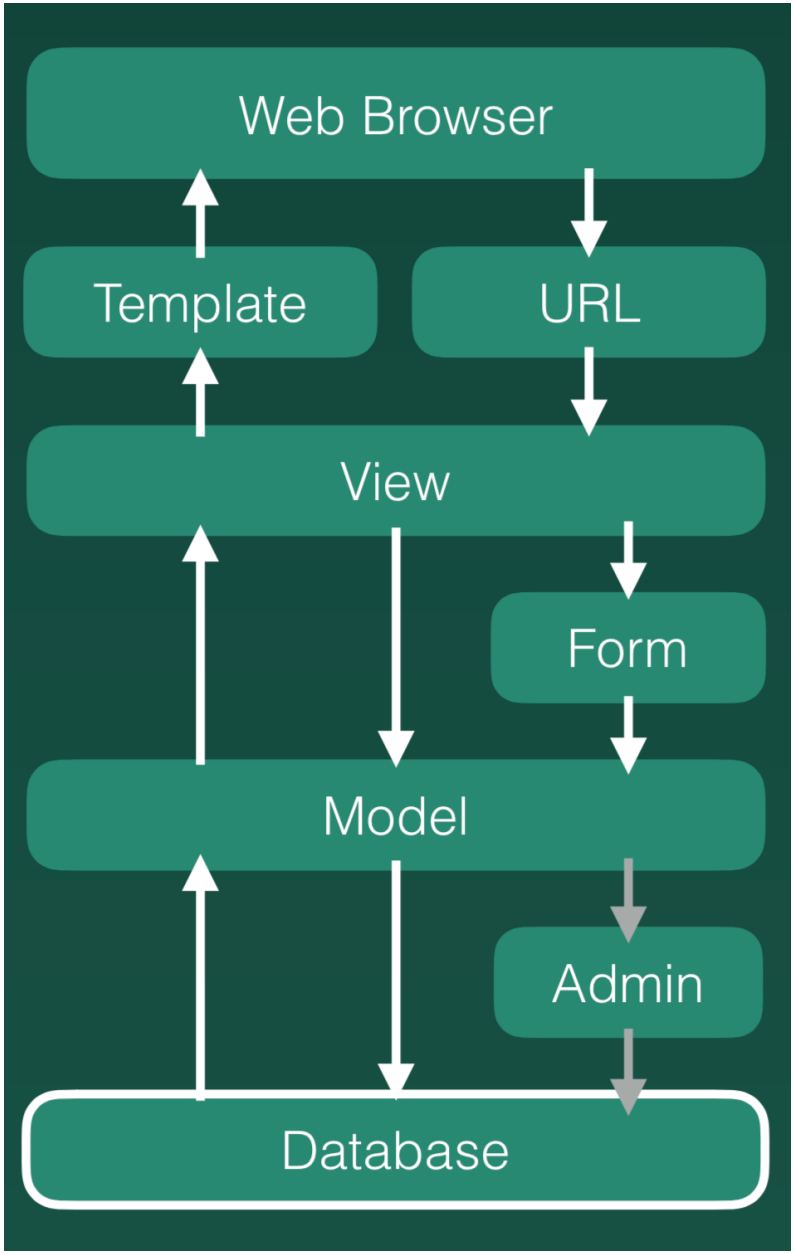
The **model-view-controller** architecture is a way to organize an application (not limited to the web!).

- The **model** is the data structure/repository.
- The **view** is the presentation/user interface.
- The **controller** is the logic that ties the model and view together and thus builds the application.



A **Model** is a class that defines a data structure that will be stored in the database.

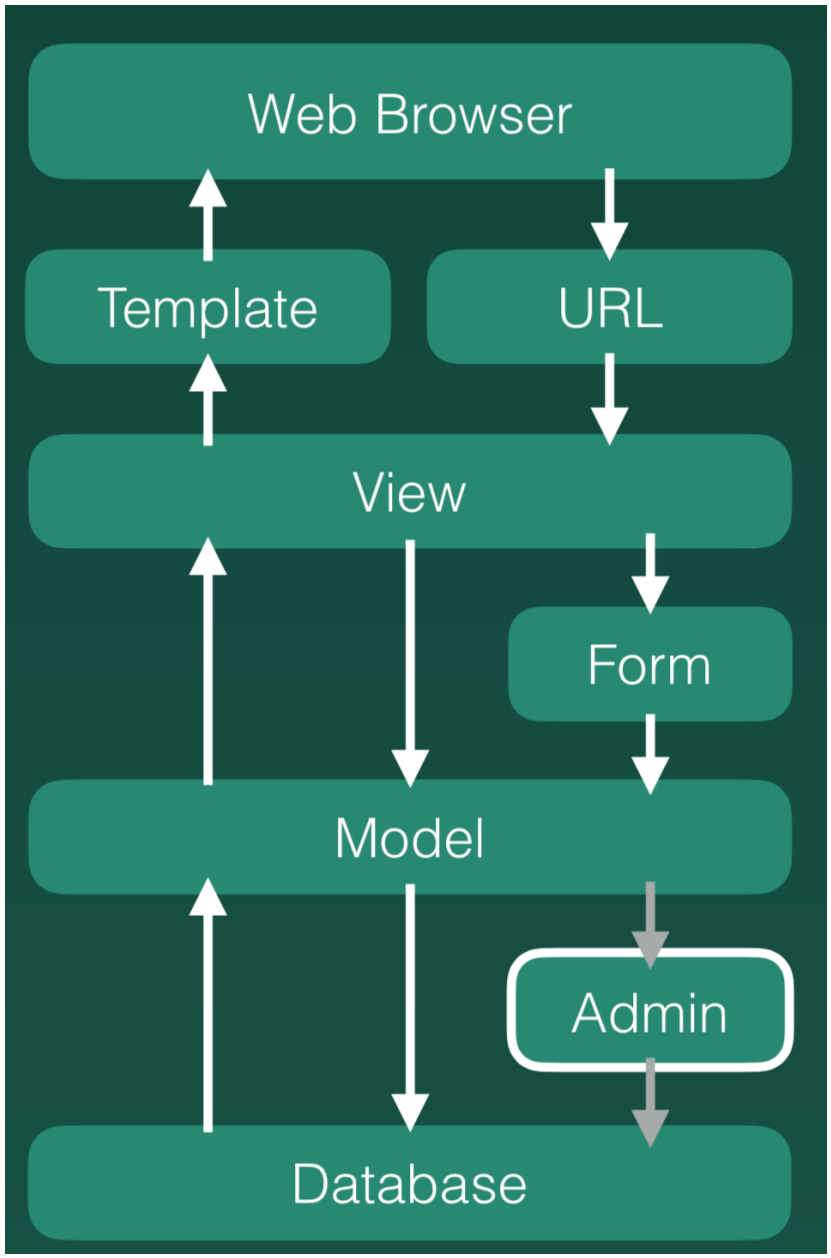
Django's **make migrations** tool will automatically create database tables to store records that follow the model's class.



The database provides persistent storage of data structures used in the web application.

Django abstracts the database administration as Python class design.

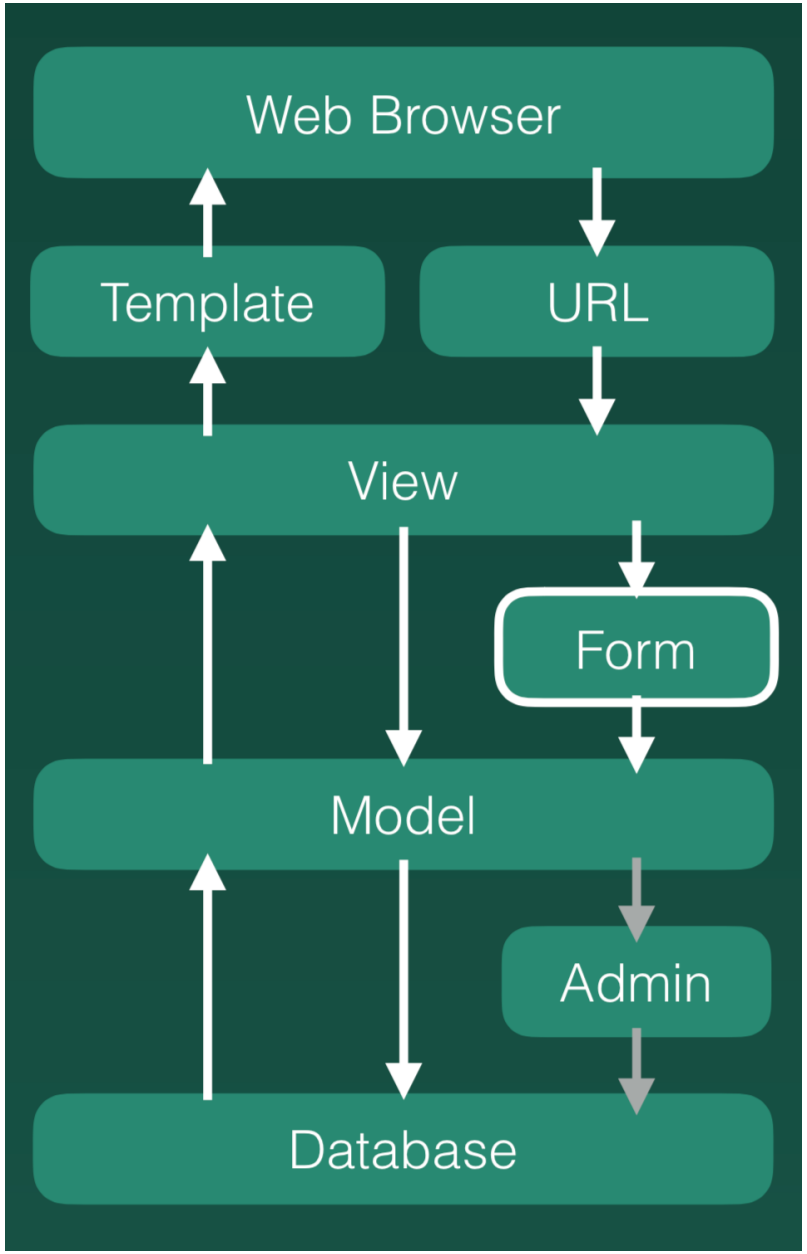
BOSTON UNIVERSITY



The **Admin** is a built-in web application that enables us to create, retrieve, update, or delete records in the database.

A single line of code is required to enable this rich administration tool.

BOSTON UNIVERSITY

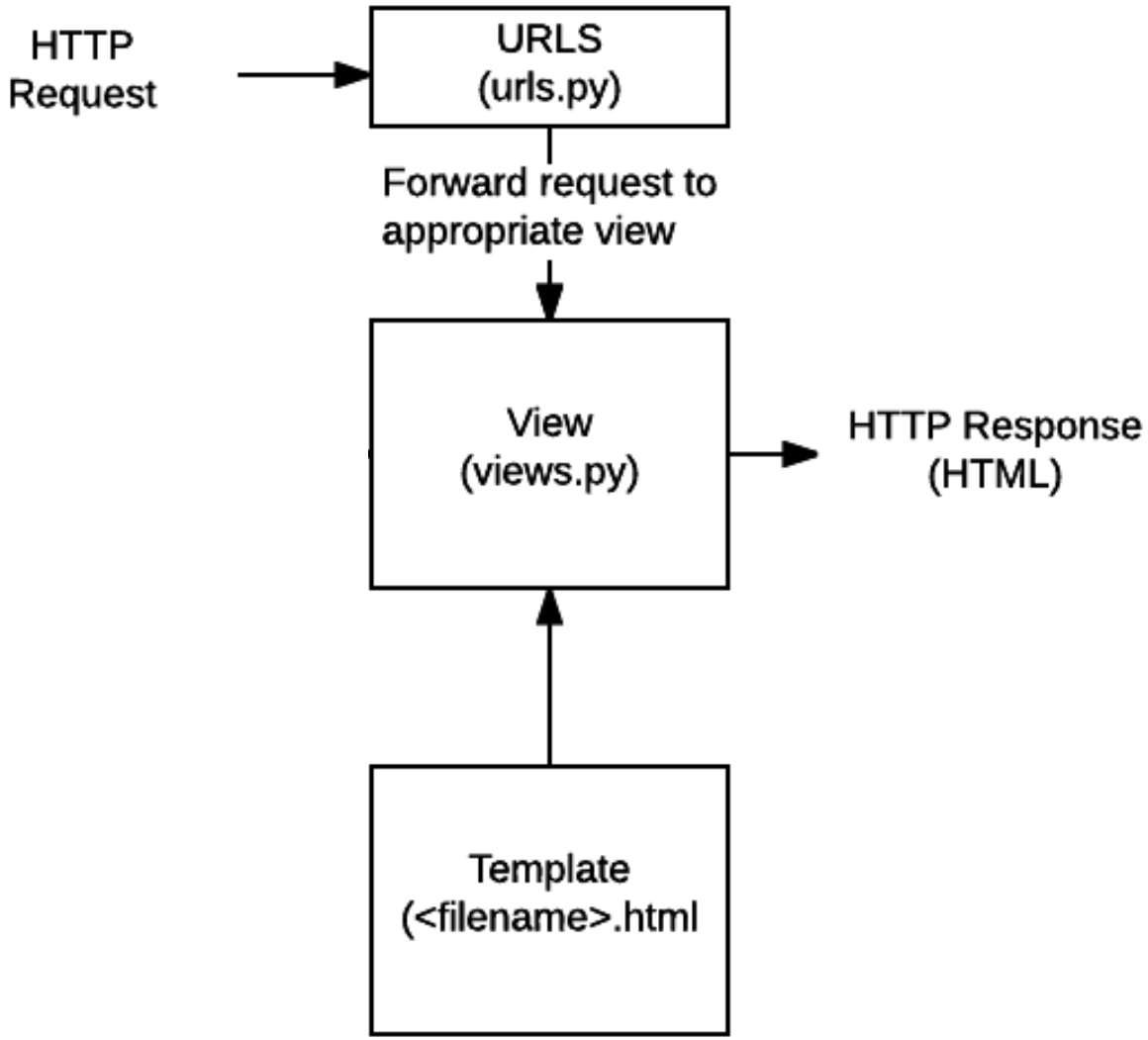


A **Form** is a Python class that maps user inputs to the data structures stored in the database.

The form is a specification for creating HTML forms.

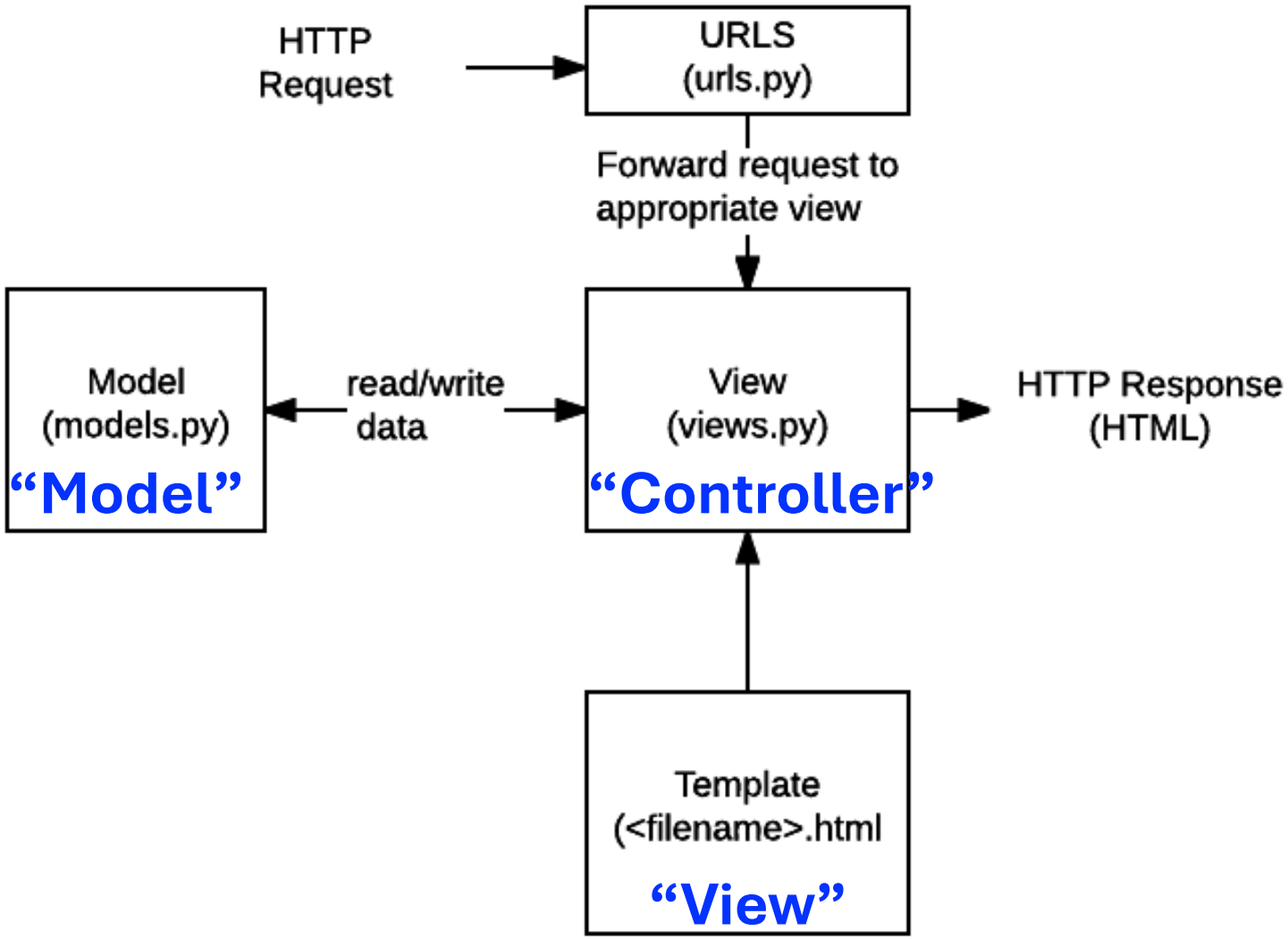
BOSTON UNIVERSITY

The Workflow of a Django Web Application



BOSTON
UNIVERSITY

The Workflow of a Django Web Application



Note the possible confusion between the terminology in the MVC architecture and the names of Django components/layers.

BOSTON UNIVERSITY

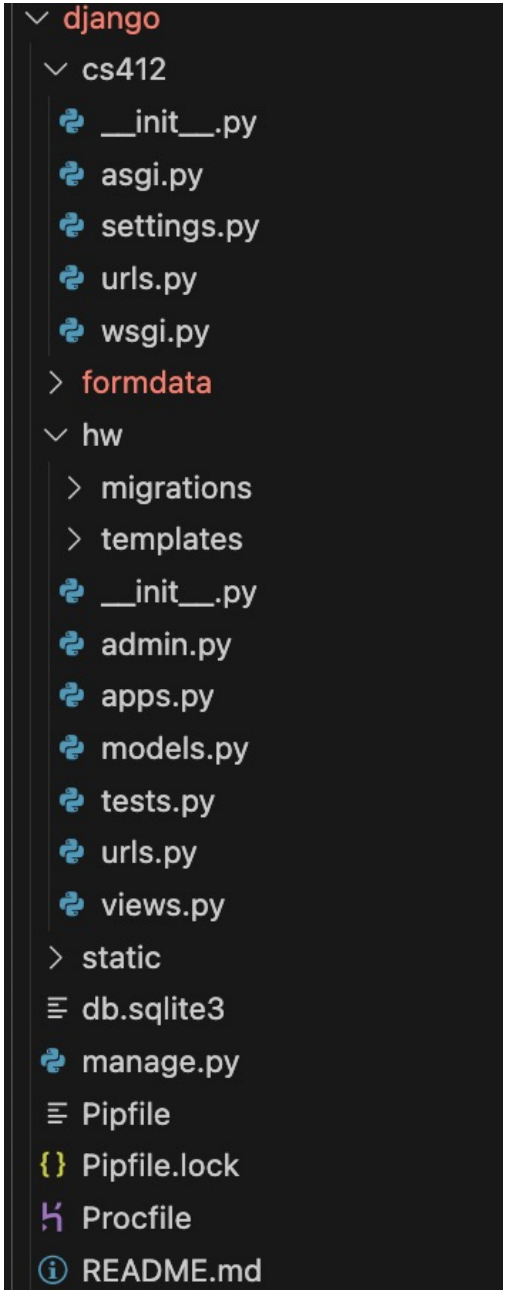
Django Directory Structure

Computer Science 412
Boston University

Aaron Stevens

BOSTON UNIVERSITY

The Django Directory Structure



The Django directory contains configuration, scripts, and subdirectories for each app.
The project directory contains configurations project-level URLs and settings.

The project-level `settings.py` file contains configurations settings.

The project-level `urls.py` file contains URL mapping for the entire project.

The `templates` directory contains HTML template files used by the generic views.

The app-level `urls.py` file contains URL-to-view mappings for one app.

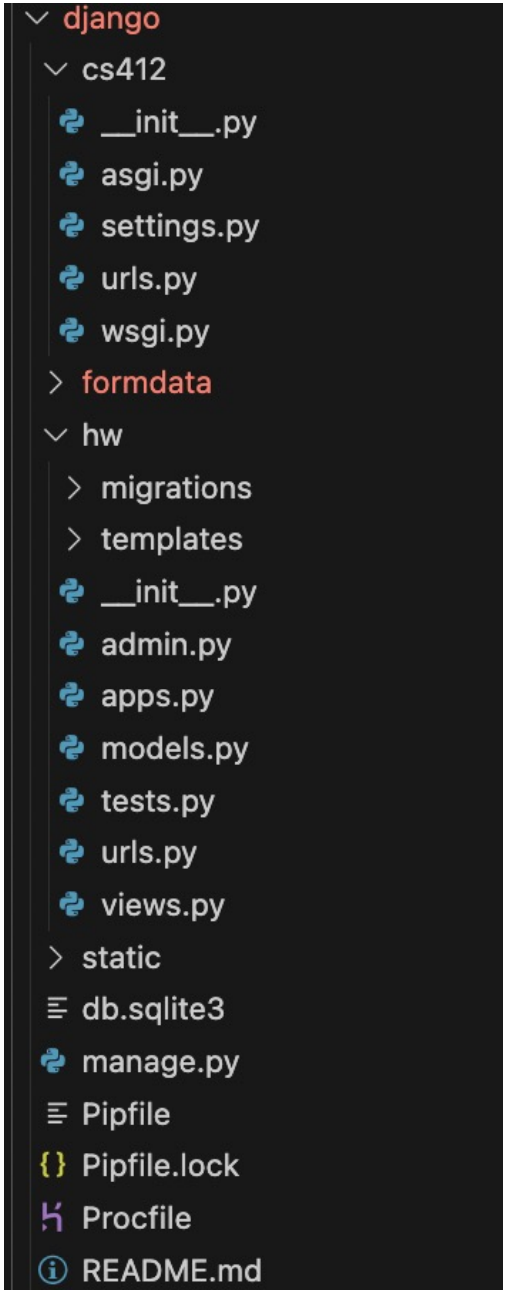
The app-level `views.py` file contains view functions or classes to respond to a URL request.

The `manage.py` file is used to run the development server, as well as administrate commands.

The `Pipfile` contains the requirements of the virtual environment.

BOSTON UNIVERSITY

The Django Directory Structure



The Django directory contains configuration, scripts, and subdirectories for each app.
The project directory contains configurations project-level URLs and settings.

The project-level `settings.py` file contains configurations settings.

The project-level `urls.py` file contains URL mapping for the entire project.

The `migrations` directory tracks changes in the database.

The `templates` directory contains HTML template files used by the generic views.

The `admin.py` file is used to register model classes with the built-in Django admin tool.

The `models.py` file is used to create model classes, which represent data objects in Django.

The app-level `urls.py` file contains URL-to-view mappings for one app.

The app-level `views.py` file contains view functions or classes to respond to a URL request.

The `db.sqlite3` file contains the database used by Django to store data records.

The `manage.py` file is used to run the development server, as well as administrate commands.

The `Pipfile` contains the requirements of the virtual environment.

BOSTON UNIVERSITY

Models describe Data Objects

A Model is a specification for a data object which will be stored persistently for our application.

- Models are Python classes which inherit from the superclass **`django.db.models.Model`**.
- A Model describes the data attributes of a record.
  - Data attributes of a model must be declared to be specific types, e.g., **`models.TextField`** or **`models.DateTimeField`**.
- Models also implement methods.
- Each unique instance of a Model object is stored as a unique database record.

Field types

Each field in your model should be an instance of the appropriate `Field` class. Django uses the field class types to determine a few things:

- The column type, which tells the database what kind of data to store (e.g. `INTEGER`, `VARCHAR`, `TEXT`).
- The default HTML widget to use when rendering a form field (e.g. `<input type="text">`, `<select>`).
- The minimal validation requirements, used in Django's admin and in automatically-generated forms.

Django ships with dozens of built-in field types; you can find the complete list in the model field reference. You can easily write your own fields if Django's built-in ones don't do the trick; see Writing custom model fields.

Django Field Types

AutoField

class* AutoField(options*)[source]**

An **IntegerField** that automatically increments according to available IDs. You usually won't need to use this directly; a primary key field will automatically be added to your model if you don't specify otherwise. See Automatic primary key fields.

BigAutoField

class* BigAutoField(options*)[source]**

A 64-bit integer, much like an **AutoField** except that it is guaranteed to fit numbers from **1** to **9223372036854775807**.

BigIntegerField

class* BigIntegerField(options*)[source]**

A 64-bit integer, much like an **IntegerField** except that it is guaranteed to fit numbers from **-9223372036854775808** to **9223372036854775807**. The default form widget for this field is a **TextInput**.

BOSTON UNIVERSITY

Django Field Types

DateField

***class* DateField(*auto_now=False*, *auto_now_add=False*, ***options*)[source]**

A date, represented in Python by a **datetime.date** instance. Has a few extra, optional arguments:

DateField.auto_now

Automatically set the field to now every time the object is saved. Useful for "last-modified" timestamps. Note that the current date is *always* used; it's not just a default value that you can override.

The field is only automatically updated when calling **Model.save()**. The field isn't updated when making updates to other fields in other ways such as **QuerySet.update()**, though you can specify a custom value for the field in an update like that.

DateField.auto_now_add

Automatically set the field to now when the object is first created. Useful for creation of timestamps. Note that the current date is *always* used; it's not just a default value that you can override. So even if you set a value for this field when creating the object, it will be ignored. If you want to be able to modify this field, set the following instead of **auto_now_add=True**:

- For **DateField**: **default=date.today** - from **datetime.date.today()**
- For **DateTimeField**: **default=timezone.now** - from **django.utils.timezone.now()**

The default form widget for this field is a **TextInput**. The admin adds a JavaScript calendar, and a shortcut for "Today". Includes an additional **invalid_date** error message key.

The options **auto_now_add**, **auto_now**, and **default** are mutually exclusive. Any combination of these options will result in an error.

BOSTON UNIVERSITY

Django Field Types

TextField

class* TextField(options*)[source]**

A large text field. The default form widget for this field is a **Textarea**.

If you specify a **max_length** attribute, it will be reflected in the **Textarea** widget of the auto-generated form field. However it is not enforced at the model or database level. Use a **CharField** for that.

URLField

***class* URLField(*max_length=200*, ***options*)[source]**

A **CharField** for a URL, validated by **URLValidator**.

The default form widget for this field is a **TextInput**.

Like all **CharField** subclasses, **URLField** takes the optional **max_length** argument. If you don't specify **max_length**, a default of 200 is used.

Example: Blog Application

We will create a Blog application.

The main data object of this application is an `Article`.

```
class Article(models.Model):
    '''Encapsulate the idea of a Article by some author.'''

    # data attributes of a Article:
    title = models.TextField(blank=False)
    text = models.TextField(blank=False)
    author = models.TextField(blank=False)
    published = models.DateTimeField(auto_now=True)
```

Migrations

A migration is a set of instructions to modify the database.

- When we make changes to our Models, the Django **`makemigrations`** command will discover changes and create a migration file to keep track of them.

```
python manage.py makemigrations <appname>
```

- We can apply (or unapply) migrations to modify the database itself, using the **`migrate`** command.

```
python manage.py migrate
```

The makemigrations Workflow

When you run **`makemigrations`**, Django inspects your models and generates a migration file:

```
python manage.py makemigrations blog
Migrations for 'blog':
  blog/migrations/0001_initial.py
    - Create model Article
```

The migration file records what changed. Every time you modify a model, run **`makemigrations`** again to capture new changes, then apply them:

```
python manage.py migrate
```

Rule of thumb: edit models.py → makemigrations → migrate

The Django Admin

The Django admin tool provides a built-in interface for CRUD operations on models: Create, Read, Update, Delete.

First, create an admin account:

```
python manage.py createsuperuser
```

Then visit the admin at: `http://127.0.0.1:8000/admin`

Registering Models with the Admin

- Django Admin only manages models that have been explicitly registered
- Registration goes in the app's `admin.py` file — one call per model
- After saving, the model appears in the Admin sidebar at `/admin`

```
# blog/admin.py
from django.contrib import admin
from .models import Article

admin.site.register(Article)
```

The `__str__()` Method

- The **`__str__()`** method a string representation for our object.
- Without it, the Django Admin shows **`Article object (1)`** — hard to identify records
- With it, the Admin shows the value you return — e.g., the article title and author

```
class Article(models.Model):
    title     = models.TextField(blank=False)
    text      = models.TextField(blank=False)
    author    = models.TextField(blank=False)
    published = models.DateTimeField(auto_now=True)

    def __str__(self):
        '''Return a string representation of this Article.'''
        return f'{self.title} by {self.author}'
```

Generic Views

A generic view is a class which implements some common features, such as:

- Rendering a template
- Showing a listing of all records of a model
- Showing a single instance of a model
- Creating a new instance of a model
- Updating an instance of a model
- Deleting an instance of a model

Why Use Generic Views?

Almost every view follows the same pattern: query the database, pass results to a template, return a response. **Generic views** encode that pattern so you don't repeat it.

Configure a generic view with class attributes instead of writing logic:

```
model                # which model to query
template_name        # which HTML template to render
context_object_name  # variable name in the template
```

Wire the view into `urls.py` using `.as_view()`:

```
path('articles/', ShowALlView.as_view(), name='list'),
path('articles/<int:pk>/', ArticleDetailView.as_view()),
```

Generic `ListView` and `DetailView`

```
 8  # Create your views here.
 9  class ShowAllView(ListView):
10      '''Define a view class to show all blog Articles.'''
11
12      model = Article
13      template_name = "blog/show_all.html"
14      context_object_name = "articles"
15
16  class ArticleView(DetailView):
17      '''Display a single article.'''
18
19      model = Article
20      template_name = "blog/article.html"
21      context_object_name = "article" # note singular variable name
```

Summary

- Models are Python classes that map to database tables; each field becomes a column
- Common field types: CharField, TextField, IntegerField, DateTimeField, BooleanField
- Migrations translate model changes into SQL: makemigrations creates, migrate applies
- Django Admin (admin.py) gives a built-in web UI for managing database records
- Generic views (ListView, DetailView) reduce boilerplate for displaying model data